

Classificação Avançada: redes neurais, SVM e ensembles

Classificação por Redes Neurais

As redes neurais artificiais são inspiradas pelos sistemas neurais biológicos, oferecendo uma abordagem poderosa para problemas de classificação complexos. Esses modelos computacionais são capazes de aprender e modelar relacionamentos não-lineares intrincados entre variáveis, tornando-os ideais para tarefas onde métodos tradicionais encontram limitações.

Inspiração Biológica

Modelam o comportamento de neurônios reais através de unidades computacionais interconectadas

Relacionamentos Não-Lineares

Capturam padrões complexos impossíveis para classificadores lineares

Aprendizado por Backpropagation

Algoritmo iterativo que ajusta pesos para minimizar erros de predição

Base do Deep Learning

Arquiteturas modernas evoluíram dessas fundações matemáticas

Arquitetura Multicamadas

Camada de Entrada

Recebe os atributos brutos dos dados, com um neurônio por variável de entrada

Camadas Ocultas

Transformam e combinam representações intermediárias; o número de camadas define a profundidade da rede

Camada de Saída

Produce a predição final; para classificação, cada neurônio representa uma classe possível

Feed-Forward

O fluxo de informação percorre a rede em uma única direção, da entrada para a saída, sem ciclos

Número de Camadas

Redes rasas possuem 1-2 camadas ocultas; redes profundas (deep) possuem dezenas ou centenas

Capacidade de Aproximação

Mesmo uma única camada oculta com neurônios suficientes pode aproximar qualquer função contínua

📖 Teorema da Aproximação Universal: com neurônios suficientes em uma camada oculta, a rede aproxima qualquer função contínua.

Neurônio: Unidade da Camada Oculta/Saída

Estrutura Matemática

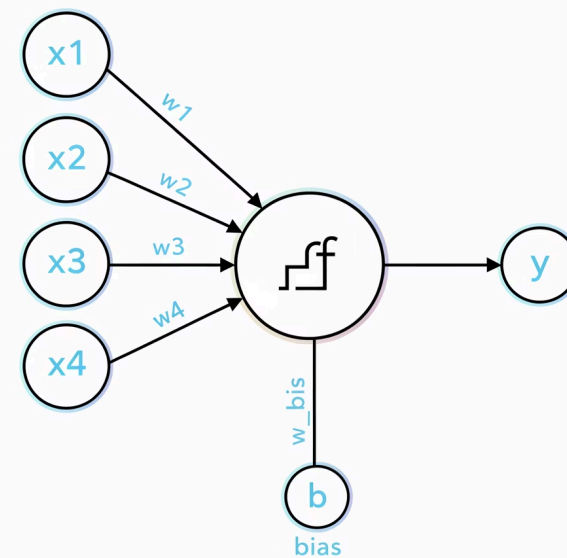
Um neurônio artificial funciona mapeando um vetor de entrada n -dimensional $\mathbf{x}$ em uma variável de saída $\mathbf{y}$ através de duas operações fundamentais:

1. **Produto Escalar Ponderado:** As entradas são multiplicadas por seus pesos correspondentes e somadas
2. **Função de Ativação Não-Linear:** O resultado é transformado por uma função que introduz não-linearidade

As entradas para cada unidade são as saídas da camada anterior. Essas entradas são multiplicadas pelos pesos correspondentes para formar uma soma ponderada, que é então adicionada ao **bias** associado à unidade. Finalmente, uma função de ativação não-linear é aplicada a esse valor acumulado.

Importância do Bias

O termo de bias permite que o neurônio desloque a função de ativação, aumentando a flexibilidade do modelo para se ajustar aos dados de treinamento.



Funções de Ativação Comuns:

- Sigmoid: $\sigma(x) = 1/(1+e^{-x})$
- ReLU: $f(x) = \max(0, x)$
- Tanh: $\tanh(x)$

Rede Neural Feed-Forward Multicamadas

Vetor de Entrada (X)

Representa os atributos dos dados de entrada, com cada dimensão alimentando um neurônio da camada de entrada

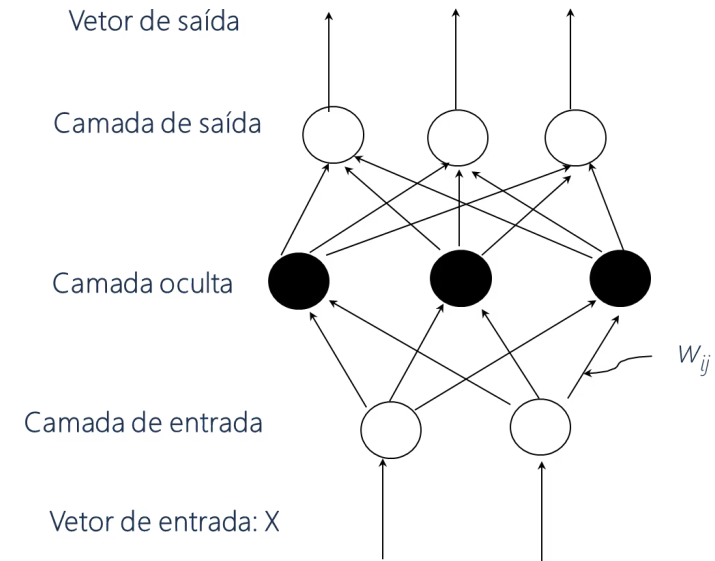
Pesos (w_{ij})

Parâmetros ajustáveis que determinam a força das conexões entre neurônios de camadas adjacentes

Vetor de Saída

Produz as previsões finais da rede, com formato dependendo do tipo de problema (classificação ou regressão)

Esta arquitetura multicamadas ilustra o fluxo de informação através da rede. A **camada de entrada** recebe os dados brutos, as **camadas ocultas** extraem representações cada vez mais abstratas, e a **camada de saída** gera a previsão final. Os pesos **w_{ij}** entre as camadas são otimizados durante o treinamento através do algoritmo de backpropagation.



Treinamento: Backpropagation

A configuração da topologia da rede neural é uma etapa crítica que influencia o desempenho do modelo. Decisões inadequadas podem resultar em **underfitting** ou **overfitting** dos dados.

Especificar Arquitetura

Defina número de camadas, unidades de entrada, ocultas e saída

Normalizar Entradas

Normalize atributos para o intervalo [0.0—1.0] para estabilizar o treinamento

Configurar Unidades de Entrada

Crie uma unidade por valor de domínio, inicializada em 0

Definir Camada de Saída

Use uma unidade por classe (classificação) ou ajuste para regressão

Iterar e Refinar

Se a precisão for inaceitável, repita com topologia ou pesos iniciais diferentes

☐ **Dica Prática:** Comece com uma arquitetura simples e aumente gradualmente a complexidade. A validação cruzada ajuda a encontrar a topologia ideal sem overfitting.

ENTRADA: D (dados rotulados), η (taxa de aprendizado),
T (épocas), arquitetura da rede definida

PRÉ-CONDIÇÃO: pesos inicializados aleatoriamente (pequenos valores)

PASSO 1 — Forward Pass

PARA CADA tupla $(x, y) \in D$:

PARA CADA camada $l = 1$ até L :

$z^l \leftarrow W^l \cdot a^{l-1} + b^l$ # soma ponderada

$a^l \leftarrow f(z^l)$ # aplica função de ativação

$\hat{y} \leftarrow a^L$ # saída da rede

PASSO 2 — Cálculo do Erro

$\delta^L \leftarrow \hat{y} - y$ # erro na camada de saída

PASSO 3 — Backward Pass

PARA CADA camada $l = L-1$ até 1 :

$\delta^l \leftarrow (W^{l+1})^T \cdot \delta^{l+1} \odot f'(z^l)$ # propaga erro

PASSO 4 — Atualização dos Pesos

PARA CADA camada l :

$W^l \leftarrow W^l - \eta \cdot \delta^l \cdot (a^{l-1})^T$

$b^l \leftarrow b^l - \eta \cdot \delta^l$

REPETIR PASSOS 1–4 por T épocas ou até convergência

SAÍDA: W, b (pesos e biases treinados da rede)

PÓS-CONDIÇÃO: erro de treinamento minimizado via gradiente descendente

Redes Neurais como Classificadores

Fraquezas

Tempo de Treinamento Longo

O processo iterativo de ajuste de pesos pode demandar recursos computacionais significativos, especialmente para redes profundas

Parâmetros Empíricos

Topologia da rede e hiperparâmetros geralmente requerem determinação experimental através de tentativa e erro

Baixa Interpretabilidade

Difícil interpretar o significado simbólico dos pesos aprendidos e das unidades ocultas, funcionando como "caixa-preta"

Forças

Alta Tolerância a Ruído

Robustez excepcional em dados com ruído e imperfeições

Generalização

Capacidade de classificar padrões não vistos durante o treinamento

Valores Contínuos

Adequadas para entradas e saídas com valores contínuos

Sucesso Comprovado

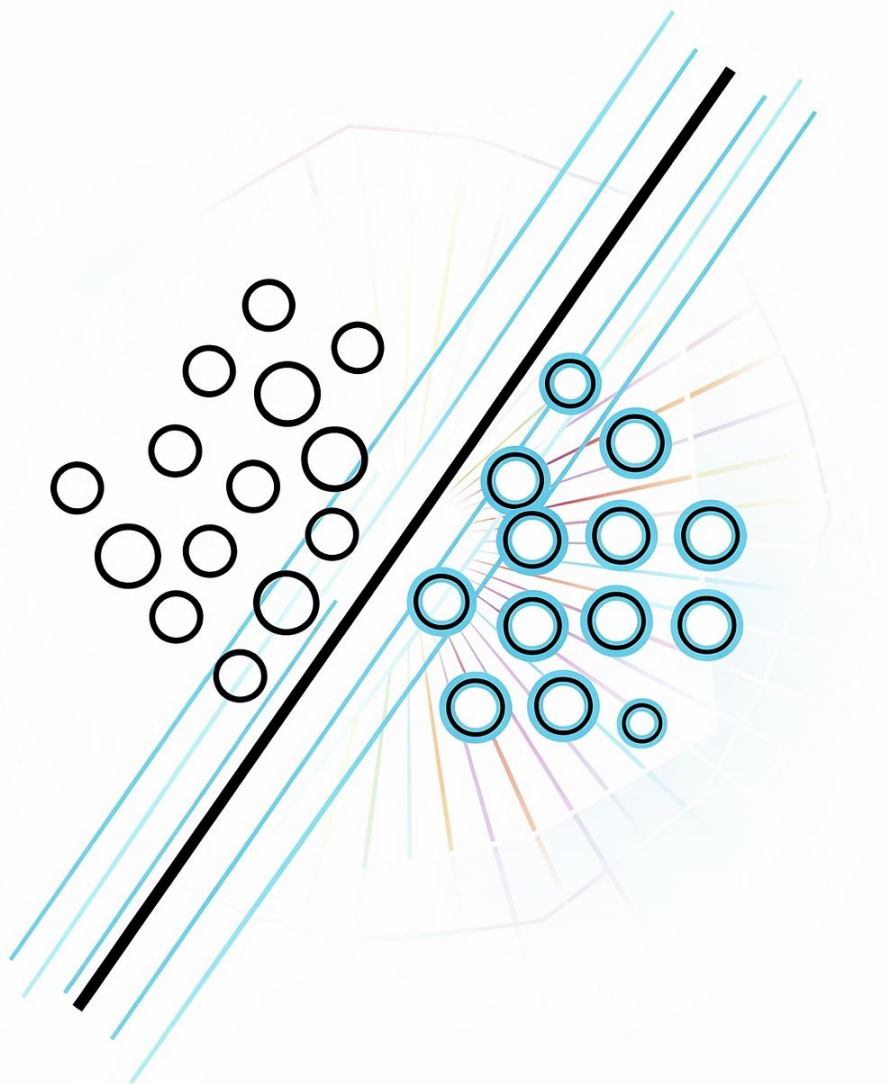
Excelente desempenho em dados reais como reconhecimento de caracteres manuscritos

Paralelização

Algoritmos inerentemente paralelizáveis para processamento eficiente

Extração de Regras

Técnicas recentes permitem extrair regras de redes treinadas



Classificação com Support Vector Machines

SVM – Support Vector Machines

Support Vector Machines (SVMs) representam um método de classificação relativamente novo, porém extremamente poderoso, capaz de lidar tanto com dados lineares quanto não-lineares. A abordagem fundamental das SVMs é transformar o problema de classificação em um espaço de maior dimensionalidade onde a separação linear se torna possível.

Mapeamento Não-Linear

Utiliza um mapeamento não-linear para transformar os dados de treinamento originais em uma dimensão superior

Busca pelo Hiperplano Ótimo

Na nova dimensão, busca o hiperplano de separação linear ótimo que serve como "fronteira de decisão"

Separação Garantida

Com mapeamento não-linear apropriado para dimensão suficientemente alta, dados de duas classes sempre podem ser separados por um hiperplano

Características Fundamentais

- **Vetores de Suporte:** Tuplas de treinamento "essenciais" que definem a fronteira de decisão
- **Margens:** Distância entre o hiperplano e os vetores de suporte mais próximos, que é maximizada
- **Efetividade em Altas Dimensões:** Mantém bom desempenho mesmo quando o número de características é muito grande

📌 **Importante:** O treinamento pode ser computacionalmente custoso para grandes conjuntos de dados. Implementações modernas frequentemente utilizam aproximações ou variantes lineares de SVM para dados em larga escala.

SVM – História e Aplicações

Origens e Desenvolvimento

Anos 1960

Vapnik e Chervonenkis desenvolvem a teoria estatística de aprendizado que fundamenta as SVMs

1992

Vapnik e colegas introduzem as Support Vector Machines modernas

Atualidade

SVMs são amplamente utilizadas em aplicações práticas de aprendizado de máquina

Características e Aplicações

Desempenho

O treinamento pode ser lento, mas a precisão é elevada devido à capacidade de modelar fronteiras de decisão não-lineares complexas através da **maximização de margem**

Versatilidade

Utilizadas tanto para **classificação** quanto para **predição numérica** (regressão)

Domínios de Aplicação

- Reconhecimento de padrões em imagens
- Classificação de texto e categorização de documentos
- Bioinformática e análise genômica
- Detecção de fraudes financeiras
- Diagnóstico médico assistido

SVM – Filosofia e Separabilidade Linear

A filosofia central das Support Vector Machines é encontrar o hiperplano de separação que **maximiza a margem** entre as classes.

Conceito de Margem Máxima

A margem é a distância perpendicular entre o hiperplano de decisão e os pontos de dados mais próximos de cada classe.

Os pontos de dados que se encontram exatamente sobre as bordas desse corredor são chamados de **vetores de suporte** - eles são os únicos pontos que realmente influenciam a posição do hiperplano de decisão.

Vantagens da Abordagem

- **Fundamentação Matemática Rigorosa:** Baseada em teoria de otimização convexa com garantias teóricas
- **Boa Generalização:** A maximização da margem reduz o risco de overfitting
- **Eficiência:** Apenas os vetores de suporte são necessários para definir o classificador final
- **Extensibilidade:** O uso de kernels permite lidar com dados não-linearmente separáveis

Quando os Dados São Linearmente Separáveis

Hiperplano de Separação

Uma fronteira linear (linha, plano ou hiperplano) que divide o espaço de atributos em duas regiões, separando completamente as classes

Vetores de Suporte

Os pontos de dados mais próximos do hiperplano de decisão, de cada classe — os únicos que definem e sustentam a fronteira de separação

Margem Máxima

A SVM seleciona o hiperplano que maximiza a distância entre as classes, garantindo a melhor separação possível e maior capacidade de generalização

SVM – Formulação e Treinamento

Formulação Matemática

Hiperplano de decisão:

$$w \cdot x + b = 0$$

Vetores de suporte satisfazem:

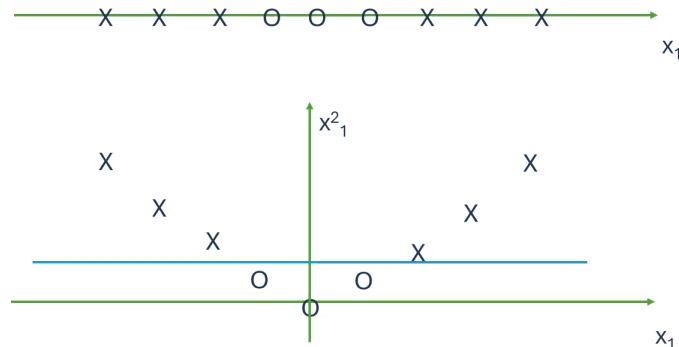
$$|w \cdot x + b| = 1$$

A margem total é $2/\|w\|$; o objetivo é minimizar $\|w\|^2$.

Propriedades Importantes

- **Solução Única:** Existe exatamente um hiperplano de margem máxima para dados linearmente separáveis
- **Dependência dos Vetores de Suporte:** Pontos distantes da margem não influenciam a solução
- **Robustez:** Pequenas perturbações que não afetam os vetores de suporte não alteram o classificador

❑ Na prática, dados perfeitamente linearmente separáveis são raros. Por isso, as SVMs utilizam **soft margins** e **kernels** para lidar com casos mais realistas.



- **2D** → Linha reta no plano
- **3D** → Plano no espaço tridimensional
- **nD** → Hiperplano de dimensão $n-1$

ENTRADA: $D = \{(x_i, y_i)\}$, C (parâmetro de margem),

K (função kernel), ϵ (tolerância)

PRÉ-CONDIÇÃO: $y_i \in \{-1, +1\}$,
atributos normalizados

PASSO 1 — Inicialização

$\alpha_i \leftarrow 0$ PARA CADA i # mult. Lagrange

$b \leftarrow 0$ # bias

PASSO 2 — Otimização (SMO)

REPETIR até convergência:

PARA CADA par (α_i, α_j) :

$E_i \leftarrow f(x_i) - y_i$ # erro

SE violação das cond. KKT:

Otimizar α_i, α_j :

$\alpha_j \leftarrow \alpha_j - y_j(E_i - E_j)/\eta$

$\alpha_j \leftarrow \text{clip}(\alpha_j, L, H)$

$\alpha_i \leftarrow \alpha_i + y_i \cdot y_j \cdot (\alpha_j - \alpha_j_{\text{old}})$

Atualizar b

PASSO 3 — Vetores de Suporte

$SV \leftarrow \{x_i \mid \alpha_i > 0\}$ # SVs = modelo

PASSO 4 — Classificação

$f(x) \leftarrow \text{senal}(\sum_i \alpha_i \cdot y_i \cdot K(x_i, x) + b)$

SAÍDA: α, b, SV

PÓS-CONDIÇÃO: hiperplano de margem máxima

SVM – Linearmente Inseparável

Na maioria dos problemas do mundo real, os dados não são linearmente separáveis no espaço de características original. As SVMs lidam com essa situação através de duas estratégias principais: **soft margins** (margens suaves) e o **kernel trick** (truque do kernel).

Soft Margins (Margens Suaves)

Permite que alguns pontos violem a margem ou mesmo sejam classificados incorretamente, introduzindo um parâmetro de penalidade **C** que controla o trade-off entre maximizar a margem e minimizar erros de classificação. Pontos que violam a margem contribuem para a função de custo.

Kernel Trick (Truque do Kernel)

Mapeia implicitamente os dados para um espaço de características de dimensão superior onde a separação linear se torna possível. O kernel permite calcular produtos internos neste espaço de alta dimensão sem explicitamente computar as coordenadas transformadas.

- ❏ **Parâmetro C:** Valores maiores de C resultam em margens mais estreitas com menos violações (risco de overfitting), enquanto valores menores permitem margens mais amplas com mais violações (melhor generalização, mas possível underfitting).

Funções Kernel para Classificação Não-Linear

Kernel Polinomial

$$K(x, y) = (x \cdot y + c)^d$$

Mapeia dados para um espaço de dimensão polinomial. O grau **d** controla a complexidade. Útil quando há interações polinomiais entre características.

Kernel RBF (Gaussiano)

$$K(x, y) = \exp(-\gamma \|x - y\|^2)$$

O kernel mais popular. Mapeia para espaço infinito-dimensional. O parâmetro γ controla a influência de cada exemplo de treinamento.

Kernel Sigmoidal

$$K(x, y) = \tanh(\alpha x \cdot y + c)$$

Similar a redes neurais com uma camada oculta. Nem sempre produz um kernel válido, mas pode ser efetivo em certos domínios.

Escolhendo o Kernel Apropriado

- **Linear:** Para dados linearmente separáveis ou de alta dimensão
- **RBF:** Boa escolha padrão quando pouco se sabe sobre os dados
- **Polinomial:** Quando se suspeita de interações polinomiais específicas

 **Validação Cruzada:** A escolha do kernel e seus parâmetros deve ser feita através de validação cruzada para evitar overfitting e garantir boa generalização.

SVM vs. Rede Neural

Support Vector Machine

Algoritmo Determinístico

Sempre produz a mesma solução para os mesmos dados, graças à formulação como problema de otimização convexa com solução única

Propriedades de Generalização

Fundamentação matemática sólida com garantias teóricas sobre generalização baseadas na teoria de aprendizado estatístico

Aprendizado em Batch

Difícil de aprender - treinamento realizado em modo batch usando técnicas de programação quadrática, que podem ser computacionalmente intensivas

Funções Complexas via Kernels

Usando kernels, pode aprender funções muito complexas mapeando implicitamente para espaços de alta dimensão

Rede Neural

Algoritmo Não-Determinístico

Diferentes inicializações de pesos podem levar a soluções distintas, pois o algoritmo pode convergir para diferentes mínimos locais

Boa Generalização Empírica

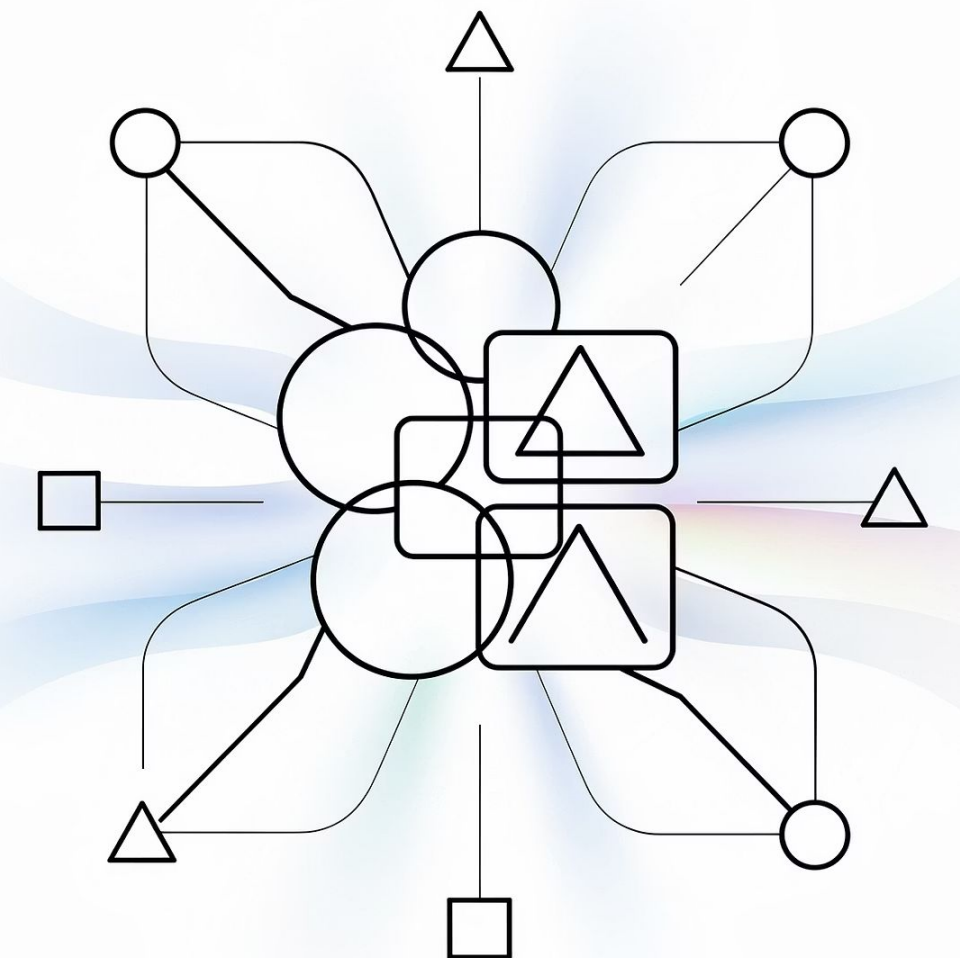
Generaliza bem na prática, mas não possui fundamentação matemática tão forte quanto SVMs para garantias de generalização

Aprendizado Incremental

Pode ser facilmente aprendida de forma incremental (online), processando exemplos um de cada vez ou em mini-batches

Funções Complexas via Profundidade

Para aprender funções complexas, use perceptron multicamadas (MLP), que é não-trivial de configurar corretamente (número de camadas, neurônios, etc.)



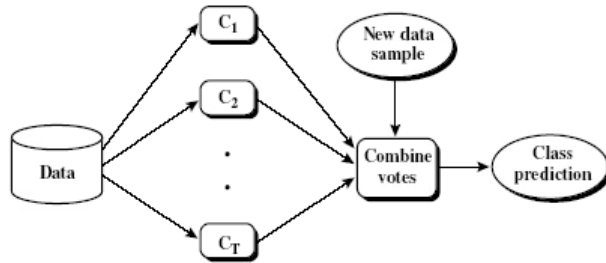
Classificação – Ensembles

Explorando técnicas avançadas de aprendizado de máquina para aumentar a precisão preditiva através de métodos ensemble e seleção inteligente de atributos.

Métodos Ensemble: Aumentando a Precisão

Os métodos ensemble combinam múltiplos modelos de aprendizado para criar previsões mais robustas e precisas do que qualquer modelo individual. Essa abordagem aproveita a diversidade entre classificadores, reduzindo variância e viés simultaneamente.

A ideia fundamental é que um grupo de "especialistas fracos" pode, quando combinado adequadamente, superar o desempenho de um único "especialista forte". Cada modelo captura diferentes aspectos dos dados, e sua agregação produz resultados mais confiáveis.



- Dong, X., Yu, Z., Cao, W., Shi, Y., Ma, Q. (2020). "A survey on ensemble learning". *Frontiers of Computer Science*, v. 14, n. 2, p. 241–258.
- Breiman, L. (2001). "Random forests". *Machine Learning*, v. 45, n. 1, p. 5–32.

Ensemble de Classificadores (Forma Geral)

ENTRADA: D (dados de treinamento), T (nº de modelos base),
 L (algoritmo base), Φ (regra de combinação)

PRÉ-CONDIÇÃO: $T \geq 2$, L definido

PASSO 1 — Construção dos Modelos Base

PARA $t = 1$ até T :

$D_t \leftarrow \text{amostrar/pesar}(D)$ # estratégia varia por método

$h_t \leftarrow L(D_t)$ # treina t-ésimo modelo base

PASSO 2 — Combinação das Predições

DADO nova instância x :

SE Φ = votação_majoritária:

$\hat{y} \leftarrow \text{argmax}_c \sum_t [h_t(x) = c]$

SE Φ = média_ponderada:

$\hat{y} \leftarrow \text{argmax}_c \sum_t \alpha_t \cdot [h_t(x) = c]$

α_t : peso do modelo t (ex: baseado em acurácia)

SE Φ = stacking:

$\hat{y} \leftarrow \text{meta_modelo}([h_1(x), \dots, h_T(x)])$

SAÍDA: $H(x) = \hat{y}$

PÓS-CONDIÇÃO: H reduz variância e/ou viés em relação a L isolado

Instâncias comuns:

Bagging $\rightarrow D_t$ por bootstrap, Φ = votação

Boosting $\rightarrow D_t$ por repesagem, Φ = média ponderada

RF $\rightarrow D_t$ por bootstrap + atributos aleatórios

Bagging: Bootstrap Aggregation

Combina modelos independentes treinados em amostras bootstrap para reduzir variância e aumentar a estabilidade do ensemble.

01

Amostragem Bootstrap

Cria múltiplos subconjuntos via amostragem com reposição do conjunto original.

03

Agregação de Previsões

Combina previsões por votação (classificação) ou média (regressão).

O bagging é particularmente eficaz para reduzir a variância de modelos de alta complexidade, como árvores de decisão profundas, tornando-os mais estáveis e menos propensos ao overfitting.

02

Treinamento Paralelo

Treina um modelo independente para cada subconjunto, sem interação entre si.

ENTRADA: D (conjunto de treinamento, n instâncias),

T (nº de modelos), L (algoritmo base)

PRÉ-CONDIÇÃO: $T \geq 1$, L definido

PASSO 1 — Geração de Modelos Base

PARA $t = 1$ até T :

$D_t \leftarrow \text{bootstrap}(D, n)$ # amostragem com reposição, $|D_t| = n$

$h_t \leftarrow L(D_t)$ # treina modelo no subconjunto

PASSO 2 — Classificação por Votação

DADO nova instância x :

PARA CADA classe c :

$\text{votos}(c) \leftarrow \sum [h_t(x) = c]$ para $t = 1..T$

$\hat{y} \leftarrow \text{argmax}_c \text{votos}(c)$ # classe com mais votos

SAÍDA: $H(x) = \hat{y}$ (predição final por votação majoritária)

PÓS-CONDIÇÃO: H combina T modelos independentes, reduzindo variância em relação a L isolado

Random Forest

Método ensemble onde cada classificador é uma árvore de decisão construída usando seleção aleatória de atributos em cada nó. Durante a classificação, cada árvore vota e a classe mais popular é retornada.

Forest-RI

Random Input Selection: Seleciona aleatoriamente F atributos como candidatos para divisão em cada nó. Usa metodologia CART para crescer árvores até o tamanho máximo.

Forest-RC

Random Linear Combinations: Cria novos atributos que são combinações lineares dos existentes, reduzindo a correlação entre classificadores individuais.

- ❑ **Vantagens:** Insensível ao número de atributos selecionados em cada divisão e mais rápido que bagging ou boosting tradicional.

ENTRADA: D (dados, n instâncias, d atributos),
 T (nº de árvores), F (nº de atributos por nó)

PRÉ-CONDIÇÃO: $F \leq d$, tipicamente $F = \lfloor \sqrt{d} \rfloor$

PASSO 1 — Construção da Floresta

PARA $t = 1$ até T :

$D_t \leftarrow \text{bootstrap}(D, n)$ # amostragem com reposição

Crescer árvore h_t em D_t :

PARA CADA nó interno:

$A_t \leftarrow \text{selecionar_aleatório}(A, F)$ # F atributos candidatos

$a^* \leftarrow \text{argmax}_{\{a \in A_t\}} \text{ganho}(a, \text{nó})$

dividir nó por a^*

Crescer até pureza máxima (sem poda)

PASSO 2 — Classificação por Votação

DADO nova instância x :

$\hat{y} \leftarrow \text{argmax}_c \sum_{t=1}^T [h_t(x) = c]$

SAÍDA: $H(x) = \hat{y}$ (votação majoritária das T árvores)

PÓS-CONDIÇÃO: H é mais estável e preciso que árvore isolada;
erro OOB estima generalização sem validação cruzada

Boosting – AdaBoost

Técnica de ensemble que constrói modelos sequencialmente, onde cada novo modelo foca em corrigir os erros dos anteriores. Diferentemente do bagging, os modelos não são independentes.

Modelo Inicial

Treina o primeiro classificador fraco com pesos uniformes

Ajuste de Pesos

Aumenta pesos das instâncias incorretas, diminui das corretas

Iteração

Treina novo modelo focando nas instâncias difíceis

Combinação Final

Agrega todos os modelos com pesos baseados em precisão

Algoritmos populares incluem **AdaBoost**, **Gradient Boosting** e **XGBoost**. O boosting é eficaz para reduzir viés, mas pode ser suscetível ao overfitting se não calibrado adequadamente.

ENTRADA: D (n instâncias rotuladas), T (nº de iterações),
 L (algoritmo base — classificador fraco)

PRÉ-CONDIÇÃO: L produz classificadores com erro < 0.5

PASSO 1 — Inicialização

$w_i \leftarrow 1/n$ PARA CADA instância i # pesos uniformes

PASSO 2 — Iteração de Boosting

PARA $t = 1$ até T :

$h_t \leftarrow L(D, w)$ # treina com pesos atuais

$\epsilon_t \leftarrow \sum_i w_i \cdot [h_t(x_i) \neq y_i]$ # erro ponderado

SE $\epsilon_t \geq 0.5$: PARAR # modelo não é melhor que aleatório

$\alpha_t \leftarrow \frac{1}{2} \cdot \ln((1 - \epsilon_t) / \epsilon_t)$ # peso do modelo

PARA CADA instância i :

SE $h_t(x_i) = y_i$: # classificada corretamente

$w_i \leftarrow w_i \cdot e^{-\alpha_t}$ # reduz peso

SENÃO: # classificada incorretamente

$w_i \leftarrow w_i \cdot e^{\alpha_t}$ # aumenta peso

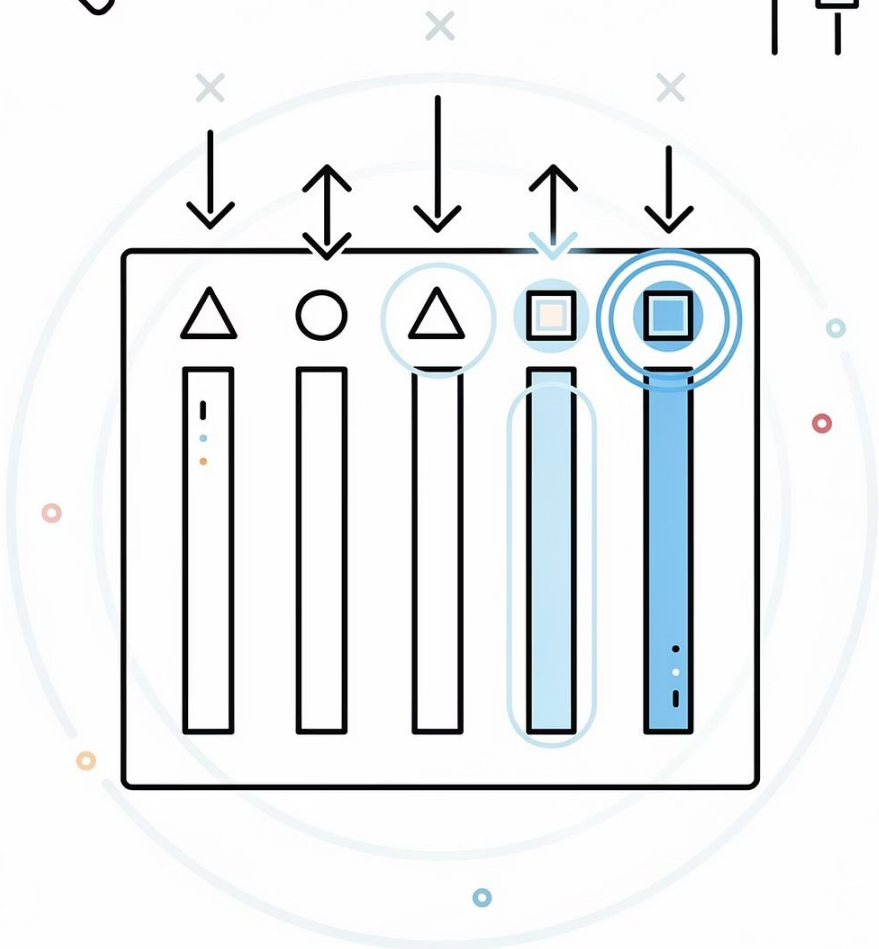
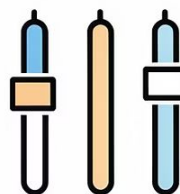
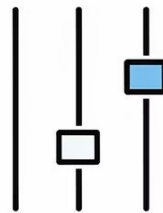
Normalizar w para $\sum w_i = 1$

PASSO 3 — Classificador Final

$H(x) \leftarrow \text{sinal}(\sum_{t=1}^T \alpha_t \cdot h_t(x))$

SAÍDA: $H(x)$ (ensemble ponderado de T classificadores fracos)

PÓS-CONDIÇÃO: H foca progressivamente nos exemplos difíceis



Classificação – Seleção de Atributos

A seleção de atributos é crucial para construir modelos eficientes e interpretáveis, eliminando redundâncias e irrelevâncias que podem prejudicar o desempenho preditivo.

Maldição da Dimensionalidade

Desafios da Alta Dimensionalidade

Esparsidade dos Dados

À medida que a dimensionalidade aumenta, os dados tornam-se progressivamente mais esparsos no espaço, dificultando a identificação de padrões significativos.

Crescimento Exponencial

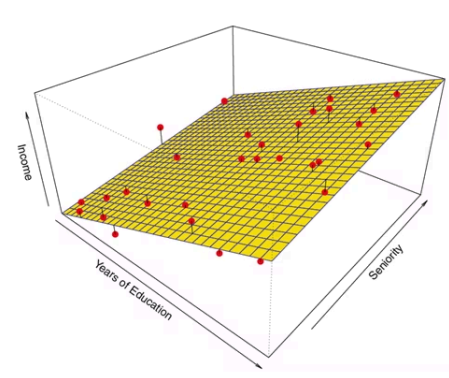
As combinações possíveis de subespaços crescem exponencialmente, tornando a exploração computacionalmente inviável.

Degradação de Métricas

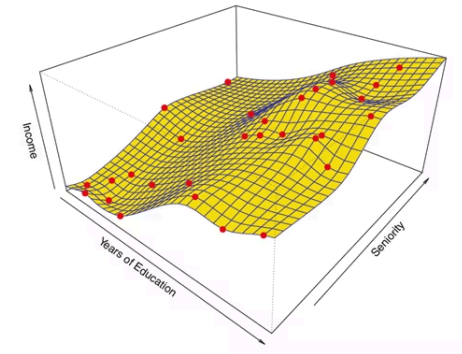
Interfere na densidade e distância entre pontos, afetando agrupamento, predição e análise de outliers de forma significativa.

Exemplo: Modelo Linear vs. Spline

Em baixas dimensões, modelos simples funcionam bem. Conforme a dimensionalidade aumenta, a flexibilidade necessária cresce drasticamente, mas os dados disponíveis se tornam insuficientes para suportar modelos complexos.



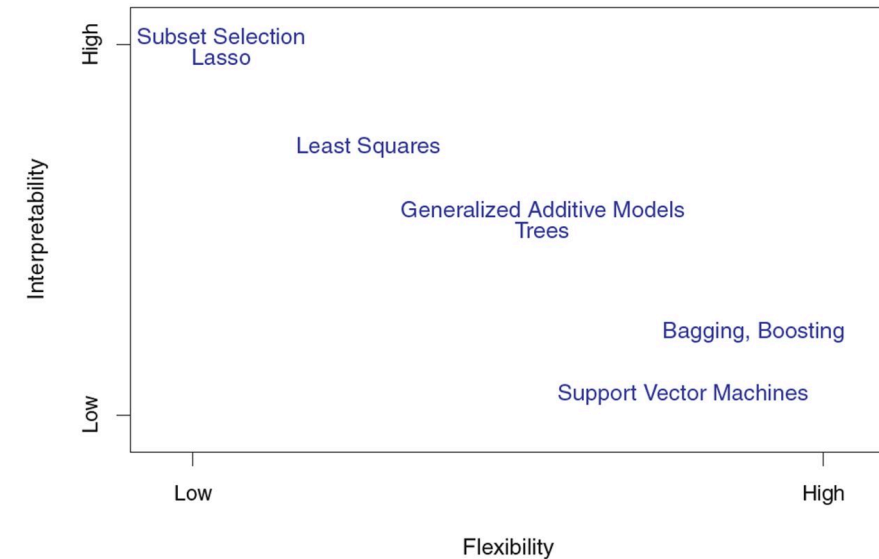
Modelo linear



Modelo spline

Interpretabilidade dos Modelos

Existe um trade-off fundamental entre flexibilidade/precisão e interpretabilidade dos modelos. Modelos mais complexos geralmente oferecem melhor desempenho preditivo, mas sacrificam a transparência e compreensão.



Alta Interpretabilidade

- Regressão linear/logística
- Árvores de decisão simples
- Modelos baseados em regras

Interpretabilidade Moderada

- GAMs (Generalized Additive Models)
- Random forests com poucas árvores
- Modelos com regularização

Baixa Interpretabilidade

- Redes neurais profundas
- Ensembles complexos
- Support Vector Machines com kernels não-lineares

Referência: James, G., Witten, D., Hastie, T., Tibshirani, R., (2013), An Introduction to Statistical Learning: with Applications in R. 1 ed. Springer.

Seleção de Atributos: Conceitos e Princípios

Atributos Redundantes

Duplicam informações contidas em um ou mais outros atributos, não agregando valor preditivo adicional ao modelo.

Exemplo: O preço de compra de um produto e o valor do imposto pago sobre essa compra são redundantes, pois o imposto é diretamente calculado a partir do preço.

A identificação e remoção desses atributos melhora a eficiência computacional, reduz overfitting e aumenta a interpretabilidade dos modelos resultantes.

A seleção de atributos segue uma arquitetura sistemática composta por etapas interdependentes, garantindo que apenas os atributos mais relevantes e informativos sejam utilizados no processo de modelagem.

Geração de Subconjuntos

Identificação e criação de candidatos a subconjuntos de atributos para avaliação.

Critério de Parada

Definição da condição de encerramento da busca, evitando exploração exaustiva do espaço de atributos.

Atributos Irrelevantes

Não contêm informações úteis para a tarefa de mineração de dados em questão, apenas adicionando ruído e complexidade desnecessária.

Exemplo: IDs de estudantes são irrelevantes para a tarefa de prever o GPA (média de notas) dos alunos, pois são apenas identificadores arbitrários sem relação com desempenho acadêmico.

Avaliação

Mensuração da qualidade de cada subconjunto com base em critérios de relevância e desempenho preditivo.

Validação

Verificação do subconjunto selecionado com dados de teste para confirmar a generalização do modelo.

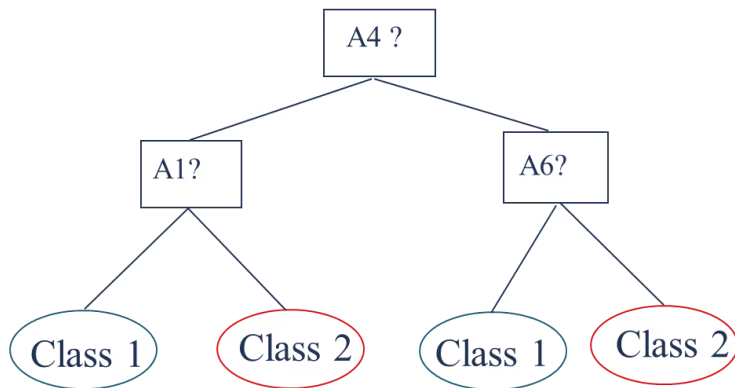
Referência: Dash, M., Liu, H., (1997), "Feature Selection for Classification", *Intell. Data Anal.*, v. 1, n. 3, p. 131–156.

Information Gain

Conjunto Inicial de Atributos

{A1, A2, A3, A4, A5, A6}

O método baseado em ganho de informação avalia cada atributo quanto à sua capacidade de reduzir a incerteza sobre a classe alvo.



Conjunto Reduzido

{A1, A4, A6}

Após a análise, apenas os atributos com maior ganho de informação são mantidos, otimizando a relação entre complexidade e poder preditivo.

Calcular Entropia Inicial

Mede a impureza do conjunto de dados antes da divisão

Avaliar Atributo A4

Calcula redução de entropia ao dividir por A4

Avaliar Atributo A1

Calcula redução de entropia ao dividir por A1

Avaliar Atributo A6

Calcula redução de entropia ao dividir por A6

Classe 1 Identificada

Atributos selecionados maximizam discriminação entre classes

Forward Stepwise Selection

Método guloso que constrói o conjunto de atributos incrementalmente, começando com conjunto vazio e adicionando um atributo por vez.

Inicialização

Começa com conjunto vazio; avalia modelo base sem preditores.

Primeira Iteração

Testa cada atributo individualmente; seleciona o de maior ganho.

Iterações Subsequentes

Avalia melhoria de cada atributo restante ao adicioná-lo ao conjunto atual.

Critério de Parada

Para quando nenhum atributo adicional melhora significativamente o desempenho.

ENTRADA: D (conjunto de treinamento), A (conjunto de atributos), θ (threshold de melhoria)
PRÉ-CONDIÇÃO: $|A| \geq 1$, modelo de avaliação definido

PASSO 1 — Inicialização
 $S \leftarrow \emptyset$ # conjunto selecionado vazio
 $perf_atual \leftarrow \text{avaliar}(\text{modelo}(S, D))$

PASSO 2 — Seleção Incremental
ENQUANTO $A \neq \emptyset$:
 $melhor_attr \leftarrow \text{NULO}$
 $melhor_perf \leftarrow perf_atual$

 PARA CADA atributo $a \in A$:
 $perf_cand \leftarrow \text{avaliar}(\text{modelo}(S \cup \{a\}, D))$
 SE $perf_cand > melhor_perf$:
 $melhor_perf \leftarrow perf_cand$
 $melhor_attr \leftarrow a$

 SE $melhor_attr = \text{NULO}$ OU $(melhor_perf - perf_atual) < \theta$:
 PARAR # critério de parada atingido

$S \leftarrow S \cup \{melhor_attr\}$ # adiciona melhor atributo
 $A \leftarrow A \setminus \{melhor_attr\}$ # remove do conjunto candidato
 $perf_atual \leftarrow melhor_perf$

SAÍDA: S (subconjunto selecionado de atributos)
PÓS-CONDIÇÃO: $|S| \leq |A_original|$, cada adição melhora perf. em $\geq \theta$

Least Absolute Shrinkage and Selection Operator (LASSO)

O LASSO opera sobre o modelo de regressão linear padrão, adicionando uma penalidade que impacta a seleção de atributos.

$$y = X\beta + \epsilon$$

Onde:

- $y \in \mathbb{R}^n$: vetor resposta
- $X \in \mathbb{R}^{n \times p}$: matriz de covariáveis
- $\beta \in \mathbb{R}^p$: vetor de coeficientes
- $\epsilon \in \mathbb{R}^n$: erro aleatório

A formulação do LASSO adiciona um termo de penalidade L_1 à função de custo da regressão:

$$\hat{\beta}^{lasso} = \arg \min_{\beta} \left\{ \frac{1}{2n} \|y - X\beta\|_2^2 + \lambda \|\beta\|_1 \right\}$$

Onde $\|\beta\|_1 = \sum_j |\beta_j|$ é a norma L_1 e $\lambda \geq 0$ é o parâmetro de regularização. Uma forma equivalente é expressa como um problema de otimização com restrição:

$$\min_{\beta} \left(\frac{1}{2n} \|y - X\beta\|_2^2 \right) \quad \text{sujeito a:} \quad \|\beta\|_1 \leq t$$

Intuição da Penalização L_1

A penalização L_1 induz esparsidade, zerando coeficientes e combinando regularização com seleção de variáveis. Geometricamente, a solução ocorre nos vértices da região de restrição L_1 .

Propriedades Chave

É um problema convexo, mas não diferenciável em $\beta_j = 0$. A solução é tipicamente encontrada via Coordinate Descent e as variáveis devem ser padronizadas para evitar que a escala influencie a penalização.

ENTRADA: X ($n \times p$), y ($n \times 1$), λ (parâmetro de regularização)

PRÉ-CONDIÇÃO: colunas de X padronizadas (média 0, var 1)

PASSO 1 — Inicialização

$\beta \leftarrow 0_p$ # coeficientes zerados

$r \leftarrow y$ # resíduo inicial = y

PASSO 2 — Iteração por Coordenada

REPETIR até convergência:

PARA CADA preditor $j = 1$ até p :

$r_j \leftarrow y - \sum_{k \neq j} x_{k,j} \cdot \beta_k$ # resíduo parcial

$\rho_j \leftarrow x_j^T \cdot r_j / n$ # correlação com resíduo

Operador de limiarização suave (soft-thresholding):

$\beta_j \leftarrow \text{sinal}(\rho_j) \cdot \max(|\rho_j| - \lambda, 0)$

$\beta_j = 0$ se $|\rho_j| \leq \lambda \rightarrow$ seleção de atributos

$\beta_j \neq 0$ se $|\rho_j| > \lambda \rightarrow$ atributo retido com encolhimento

PASSO 3 — Critério de Convergência

SE $\max_j |\Delta \beta_j| < \epsilon$: PARAR

SAÍDA: $\hat{\beta}$ (vetor esperso de coeficientes)

PÓS-CONDIÇÃO: $\hat{\beta}_j = 0$ para atributos irrelevantes (seleção automática)

Correlation-based Feature Selection (CFS)

CFS avalia subconjuntos de atributos pela correlação atributo-classe versus correlação atributo-atributo, equilibrando relevância e redundância.

Alta Correlação com Classe

Atributos fortemente correlacionados com a variável alvo, maximizando poder preditivo.

Baixa Correlação Entre Si

Atributos com baixa correlação mútua, minimizando redundância.

Função de Mérito

Métrica única que pondera relevância individual versus redundância do conjunto.

Processo de Seleção

1. Calcular correlações atributo-classe
2. Calcular correlações atributo-atributo
3. Avaliar subconjuntos usando função de mérito
4. Selecionar subconjunto com melhor pontuação

Características

- Abordagem baseada em filtro, independente do algoritmo
- Eficiente computacionalmente
- Lida naturalmente com redundância

ENTRADA: A (conjunto de atributos), C (classe alvo), D (dados)

PRÉ-CONDIÇÃO: correlações calculáveis entre atributos e classe

PASSO 1 — Cálculo de Correlações

PARA CADA atributo $a \in A$:

$$r_{ac}[a] \leftarrow \text{correlação}(a, C) \quad \# \text{ relevância com a classe}$$

PARA CADA par $(a_i, a_j) \in A \times A$:

$$r_{aa}[i,j] \leftarrow \text{correlação}(a_i, a_j) \quad \# \text{ redundância entre atributos}$$

PASSO 2 — Avaliação de Subconjuntos

PARA CADA subconjunto $S \subseteq A$:

$$k \leftarrow |S|$$

$$\text{mérito}(S) \leftarrow (k \cdot \bar{r}_{ac}) / \sqrt{(k + k \cdot (k-1) \cdot \bar{r}_{aa})}$$

$\bar{r}_{ac}$: média das correlações atributo-classe em S

$\bar{r}_{aa}$: média das correlações inter-atributos em S

PASSO 3 — Seleção

$$S^* \leftarrow \operatorname{argmax}_S \text{mérito}(S)$$

SAÍDA: S^* (subconjunto ótimo de atributos)

PÓS-CONDIÇÃO: S^* maximiza relevância e minimiza redundância

RELIEF

Algoritmo baseado em instâncias que atribui pesos aos atributos de acordo com sua relevância para distinguir entre classes diferentes.

Amostragem Aleatória

Seleciona instâncias aleatórias do conjunto de treinamento para análise iterativa.

Identificação de Vizinhos

Identifica o vizinho mais próximo da mesma classe (nearHit) e de classe diferente (nearMiss).

Cálculo de Score

Aumenta o peso se diferencia nearMiss; diminui se diferencia nearHit.

Seleção por Threshold

Seleciona atributos cujo score supera o limiar mínimo definido pelo usuário.

Pontos Fortes

- Detecta interações entre atributos
- Robusto a ruído nos dados
- Não assume independência de atributos

Considerações

- Sensível à métrica de distância utilizada
- Pode ser custoso em grandes conjuntos
- Threshold de seleção requer ajuste

Referência: Dash, M., Liu, H., (1997)

ENTRADA: D (conjunto de treinamento, n instâncias),
m (nº de amostras), τ (threshold)

PRÉ-CONDIÇÃO: atributos normalizados em [0,1],
classes definidas

PASSO 1 — Inicialização

$W[a] \leftarrow 0$ PARA CADA atributo a # pesos zerados

PASSO 2 — Iteração por Amostras

PARA $i = 1$ até m:

$x \leftarrow \text{amostra_aleatória}(D)$

$\text{nearHit} \leftarrow \text{vizinho_mais_próximo}(x, \text{mesma_classe})$

$\text{nearMiss} \leftarrow \text{vizinho_mais_próximo}(x, \text{classe_diferente})$

PARA CADA atributo a:

$W[a] \leftarrow W[a]$

- $\text{diff}(a, x, \text{nearHit})^2 / m$

penaliza se separa mesma classe

+ $\text{diff}(a, x, \text{nearMiss})^2 / m$

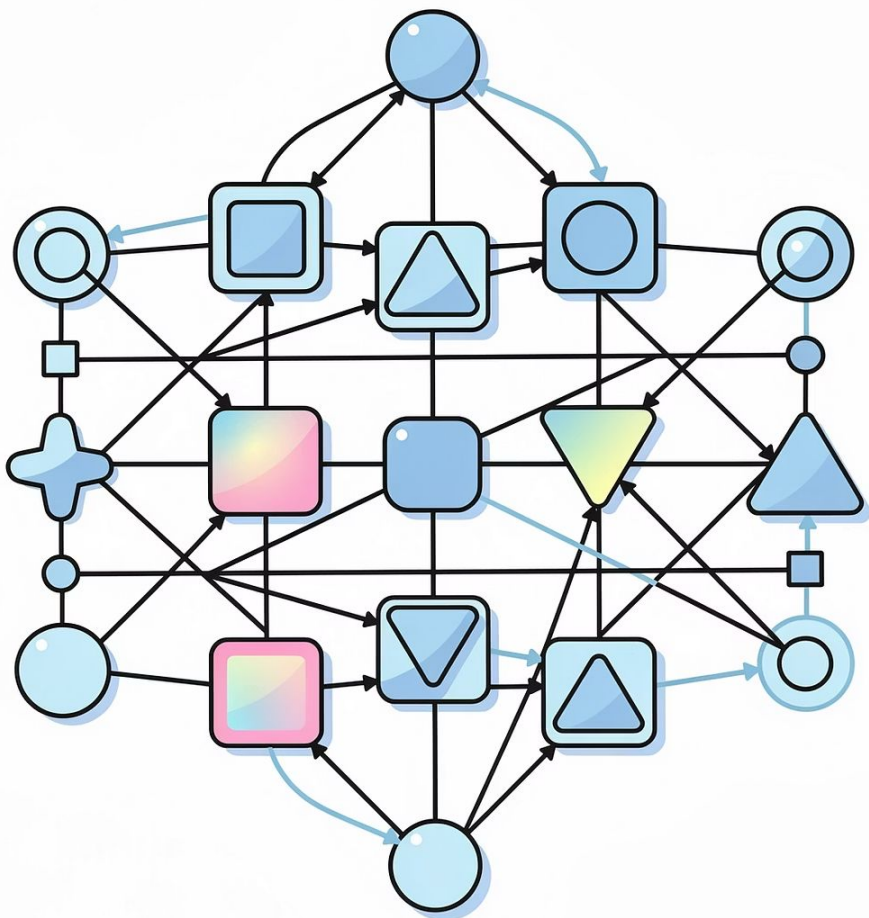
recompensa se separa classes diferentes

PASSO 3 — Seleção por Threshold

$S \leftarrow \{ a \in A \mid W[a] \geq \tau \}$

SAÍDA: S (atributos relevantes), W (vetor de pesos)

PÓS-CONDIÇÃO: atributos em S distinguem classes
melhor que o threshold τ



Tópicos Avançados

Neste módulo, exploraremos conceitos avançados em classificação de aprendizado de máquina. Abordaremos o teorema "No Free Lunch", técnicas para lidar com conjuntos de dados desbalanceados, classificação multiclasse, aprendizado semi-supervisionado e transfer learning. Esses tópicos são essenciais para desenvolver soluções robustas em cenários complexos do mundo real.

No Free Lunch em Classificação

Ausência de Superioridade Universal

Nenhum classificador único apresenta desempenho superior em todos os problemas possíveis. A eficácia de cada algoritmo varia conforme o contexto específico.

Influência das Métricas

As métricas de avaliação selecionadas exercem impacto direto na escolha do modelo mais adequado para cada aplicação.

Dependência das Características dos Dados

O desempenho está intrinsecamente ligado às características dos dados e à representação de features escolhida para o problema.

Seleção Empírica

A seleção de modelos é fundamentalmente empírica e dependente do contexto, exigindo experimentação sistemática e validação rigorosa.

Classificação de Conjuntos de Dados Desbalanceados

O Problema do Desbalanceamento

O problema de desbalanceamento de classes ocorre quando há exemplos positivos raros, mas numerosos exemplos negativos. Isso é comum em diagnóstico médico, detecção de fraudes, vazamento de óleo, detecção de falhas, entre outros.

Métodos tradicionais assumem distribuição balanceada de classes e custos de erro iguais, tornando-se inadequados para dados desbalanceados.

Oversampling

Reamostragem de dados da classe positiva para aumentar sua representação no conjunto de treinamento.

Under-sampling

Eliminação aleatória de tuplas da classe negativa para balancear a distribuição das classes.

Threshold-moving

Move o limiar de decisão para facilitar a classificação de tuplas da classe rara, reduzindo falsos negativos custosos.

Técnicas de Ensemble

Combinação de múltiplos classificadores para melhorar o desempenho geral em dados desbalanceados.

❏ O problema de desbalanceamento de classes permanece um desafio significativo em tarefas de classificação multiclasse, exigindo abordagens mais sofisticadas.

❏ *Referência:* He, H., Garcia, E. A. (2009). "Learning from imbalanced data". *IEEE Transactions on Knowledge and Data Engineering*, v. 21, n. 9, p. 1263–1284.

Classificação Multiclasse

A classificação multiclasse estende os conceitos binários para cenários onde existem três ou mais classes mutuamente exclusivas. Diferentemente da classificação binária, onde temos apenas duas categorias possíveis, a classificação multiclasse requer estratégias específicas para lidar com múltiplas categorias simultaneamente.

One-vs-Rest (OvR)

Treina um classificador binário para cada classe contra todas as outras classes. Simples de implementar, mas pode sofrer com desbalanceamento.

One-vs-One (OvO)

Treina um classificador binário para cada par de classes. Requer mais modelos, mas cada um enfrenta um problema mais simples.

Classificadores Nativos

Alguns algoritmos como redes neurais, árvores de decisão e regressão logística multinomial suportam multiclasse nativamente.

A escolha da estratégia depende do número de classes, da complexidade computacional aceitável e das características específicas do conjunto de dados. Técnicas de ensemble podem combinar múltiplas abordagens para melhorar a robustez.

Classificação Semi-Supervisionada

A classificação semi-supervisionada aproveita tanto dados rotulados quanto não rotulados para melhorar o desempenho do modelo. Esta abordagem é particularmente valiosa quando rotular dados é caro ou demorado, mas dados não rotulados são abundantes.

Dados Rotulados (labeled)

- Fornecem sinal de aprendizado claro
- Permitem validação precisa
- Requerem esforço humano significativo

Dados Não Rotulados (unlabeled)

- Abundantes e de baixo custo
- Revelam estrutura dos dados
- Melhoram fronteiras de decisão

Self-Training

O modelo rotula seus dados mais confiantes e os adiciona ao conjunto de treinamento iterativamente.

Co-Training

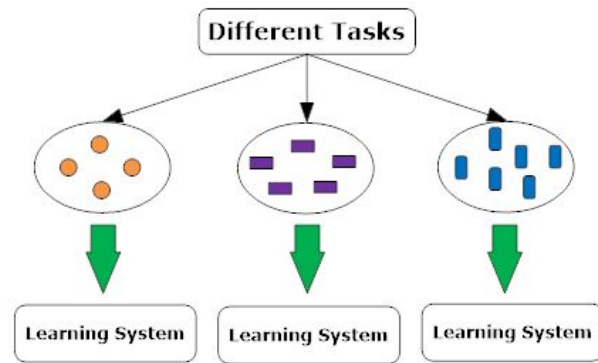
Múltiplos modelos treinados em diferentes views dos dados colaboram para rotular exemplos não supervisionados.

Graph-Based

Propaga rótulos através de grafos baseados em similaridade entre exemplos rotulados e não rotulados.

Transfer Learning: Framework Conceitual

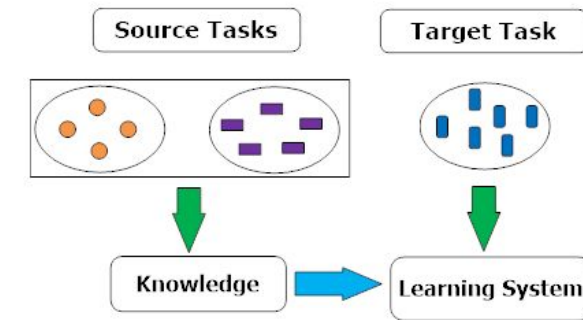
Aprendizado Tradicional



No framework tradicional, um novo classificador é construído do zero para cada nova tarefa. Isso requer grandes quantidades de dados rotulados e tempo computacional para cada problema individual.

- Cada tarefa é tratada isoladamente
- Requer dados abundantes para cada domínio
- Não aproveita conhecimento prévio

Framework de Transfer Learning



Transfer learning extrai conhecimento de uma ou mais tarefas fonte e aplica esse conhecimento a uma tarefa alvo. Constrói novos classificadores aplicando conhecimento existente aprendido de tarefas fonte.

- Reutiliza conhecimento entre domínios
- Reduz necessidade de dados rotulados
- Acelera convergência do treinamento

Transfer Learning: Métodos e Aplicações

Transfer Learning Baseado em Instâncias

Repondera alguns dados das tarefas fonte e os utiliza para aprender a tarefa alvo, permitindo transferência direta de exemplos relevantes.

Transfer AdaBoost

Assume que dados fonte e alvo são descritos pelos mesmos atributos e rótulos, mas com distribuições diferentes. Requer rotular apenas pequena quantidade de dados alvo.

Como Transfer AdaBoost Utiliza Dados Fonte

Quando uma tupla fonte é classificada incorretamente, o algoritmo reduz o peso dessas tuplas para que tenham menor efeito nos classificadores subsequentes.

Questões de Pesquisa em Aberto

Transferência Negativa

Ocorre quando o modelo com transferência apresenta desempenho inferior ao modelo sem transferência, geralmente devido a diferenças significativas entre domínios.

Transfer Learning Heterogêneo

Transferência de conhecimento entre espaços de features diferentes ou múltiplos domínios fonte com características distintas.

Transfer Learning em Larga Escala

Desenvolvimento de métodos eficientes para transferir conhecimento com grandes volumes de dados e modelos complexos como deep learning.

📖 *Referência:* Weiss, K., Khoshgoftaar, T. M., Wang, D. D. (2016). "A survey of transfer learning". *Journal of Big Data*, v. 3, n. 1.

Referências Principais

Esta seleção de referências representa os pilares fundamentais para o estudo aprofundado de mineração de dados, cobrindo desde conceitos básicos até técnicas avançadas e aplicações contemporâneas.



1. **J. Han, J. Pei, and H. Tong**, *Data Mining: Concepts and Techniques*, 4th edition. Cambridge, MA: Morgan Kaufmann, 2022.
2. **G. M. James, D. Witten, T. Hastie, and R. Tibshirani**, *An Introduction to Statistical Learning: With Applications in R*. Springer Nature, 2021.
3. **S. Garcia, J. Luengo, and F. Herrera**, *Data Preprocessing in Data Mining*. Springer, 2014.
4. **C. M. Bishop and H. Bishop**, *Deep Learning: Foundations and Concepts*. Springer Nature, 2023.